

PyArchInit

What it is, what it does, and why it matters for an archaeologist

Enzo Cocca (CNR-ISPC / adArte / DISCI-UNIBO)

May 2026

Contents

PyArchInit — What it is, what it does, and why it matters for an archaeologist	1
1. What PyArchInit is, in one paragraph	1
2. The idea behind it	2
3. Who it is for	2
4. What it covers	2
5. What you can obtain from it	3
6. Beyond the core: analysis, animation, site management	5
7. The compromises (the honest part)	6
8. What PyArchInit is not	7
9. What PyArchInit talks to	7
10. One-page summary	7
Where StratiGraph fits	8

PyArchInit — What it is, what it does, and why it matters for an archaeologist

Field	Value
Project	StratiGraph (Horizon Europe)
Partner	3D Research / adArte Srl
Document type	High-level functional description (narrative SRS)
Target reader	Archaeologist, project officer, scientific reviewer
Version	3.0
Date	May 2026
Author	Enzo Cocca (CNR-ISPC / adArte / DISCI-UNIBO)

This document describes PyArchInit at the level of *what it does* for the archaeologist, not *how* it does it. There are no step-by-step instructions, no menus to click, no code. The purpose is to give the reader an honest map of what the system covers, what it produces, the compromises it asks for, and where its limits are. On top of this map, technical documentation, tutorials and developer guides can be built separately, and will evolve over time without invalidating what is written here.

1. What PyArchInit is, in one paragraph

PyArchInit is an archaeological information system that lives inside QGIS. In practice it is a plugin: you install QGIS, you install PyArchInit on top of it, and from that moment your GIS stops being only a map and becomes also an excavation archive. All the records that an archaeologist normally fills in on paper or in a spreadsheet —

site sheets, stratigraphic units, finds, structures, periods, photos, drawings — are kept in a single database that is automatically linked to the cartography. The aim is simple: every drawn polygon corresponds to a real record, every record can be found on the map, and nothing has to be re-entered twice.

PyArchInit was born in 2005 from the work of Luca Mandolesi and has grown since then into a community project, hosted on GitHub and distributed through the official QGIS plugin repository. It is open source, free, and used in academic, professional and institutional contexts in Italy and abroad. It is not a product sold by a company: it is a tool maintained by archaeologists who also develop software.

In short: a single working environment where the stratigraphy, the materials, the photos, the drawings and the map of an excavation live together, talk to each other, and can be queried as a whole.

2. The idea behind it

Archaeological documentation has a structural problem: the same information is recorded many times, in different places, by different people, with different conventions. The paper US form, the photo log, the find inventory, the field drawing, the digitised plan in CAD or GIS, the final report — every step risks reintroducing errors or losing the link between the object and its context.

PyArchInit takes a precise position on this problem: the primary unit is not the document, it is the **record in the database**. The site sheet, the US, the find, the structure are rows in tables. Maps, photos, matrices and PDF sheets are **views** of those rows. Change the row and everything downstream updates. Delete a relationship and the Harris Matrix changes accordingly. Reclassify a US to a different period and the periodised plan reflects it the next time you open it.

This is the most important conceptual point of the system, and it is the reason for almost every design choice that follows: **the data is the source of truth; everything else is produced from it.**

3. Who it is for

PyArchInit is built for the people who actually do excavation documentation. Concretely:

- **The field archaeologist** who needs to record US sheets, attach photos and drawings, mark finds, and not lose anything between the trench and the lab.
- **The site director / scientific responsible** who needs to keep the overall picture coherent: periodisation, structures, validation, final reports.
- **The materials specialist** who works with the finds inventory, the typologies, the quantifications, and needs to cross-reference materials with their stratigraphic context.
- **The GIS / data manager** who keeps the database healthy, manages the projection systems, the backups, the imports from external sources, the team accounts on the server.
- **The reviewer in the Soprintendenza or in an institution** who receives the catalogue and expects an ICCD-compatible deliverable, not a custom format that nobody else uses.

PyArchInit is not for casual users who just want to draw on a map. The investment of learning it makes sense when there is real archaeological data to manage, a real archive to keep alive, and real deliverables to produce.

4. What it covers

PyArchInit is organised internally around a set of **schede** (forms), each corresponding to one of the objects that an archaeologist routinely records. These are not pages of a website: they are the categories around which archaeological documentation is built. The main ones are:

Area	What you can do with it
Site (Scheda Sito)	Define the site, its location, the responsible, the type of investigation, the chronology. This is the top-level container; everything else hangs from here.

Area	What you can do with it
Stratigraphic Units (US / USM)	Record every stratigraphic unit with description, interpretation, definition, formation, components, and — most importantly — its stratigraphic relations with other units (covers, is covered by, cuts, is cut by, fills, abuts, equals, and the rest of the standard set).
Periodisation	Define the periods and phases of the site and assign units to them. From this you get period-based queries, period-coloured plans, and the matrix grouped by phase.
Structures	Group US/USM into structures (a wall, a building, a tomb chamber, an entire phase of a complex). Useful when the unit of analysis is bigger than the single stratigraphic unit.
Finds inventory (Reperti / RA)	Catalogue every find with provenance, class, type, state of conservation, dating, position. Each find is linked to its US and inherits its context automatically.
Materials / pottery (incl. TMA)	Specialised cataloguing for ceramic, lithic, glass, metal and organic materials, with the ICCD-compliant TMA (Tabella Materiali Archeologici) for container-based registration, quantitative fields, and the possibility to run quantifications (NMI, weights, fragment counts) on whatever subset you query.
Taphonomy / burials	Specific forms for the documentation of burials, individuals, and taphonomic observations, where the excavation has this dimension.
Multimedia	Photos, drawings, 3D models, documents linked to any record. Once attached, a photo of a US is reachable from the US sheet, from the find that came from that US, and from the map.
Thesaurus	Controlled vocabularies for the descriptive fields. Reduces free-text noise, makes queries actually work, and aligns terminology across operators.
UT — Topographic Units	For landscape archaeology and survey: documentation of topographic units at a scale broader than the single excavation.
Spatial layers	A set of geographic layers (US polygons, finds, elevations, section lines, reference points, structural hypotheses) that mirror the database and let the archaeologist work on the map as naturally as on the form.

All of these areas are interconnected. The site contains the units, the units carry the relations, the relations build the matrix, the matrix feeds the periodisation, the periodisation drives the plans, the finds are anchored to the units, the media to everything. This is the value of the system: not the individual form, but the connections between them.

5. What you can obtain from it

This is the section that matters most for a reader who has to decide whether PyArchInIt fits a project. The list below is concrete: things that come out of the system as a deliverable, an output, or an answer to a question.

5.1 The Harris Matrix, built for you

You record the stratigraphic relations one by one on the US sheets. PyArchInit, on demand, reads all of them and draws the Harris Matrix automatically. You do not draw it by hand, and you do not have to keep it consistent by hand.

More importantly: the system **checks the relations**. If you say that US 5 covers US 7, but on US 7 you forgot to write that it is covered by US 5, PyArchInit detects the asymmetry. The classical “inverse relations” problem — the single most common source of errors in stratigraphic documentation — is handled by the system. It tells you what is missing, what is contradictory, what is dangling, and lets you fix it before the matrix is generated.

The matrix can be exported as a graphic file (PDF, PNG, SVG) and can be grouped by period and phase, so what you publish is not the raw graph but the interpreted one.

5.2 Periodised plans and thematic maps

Because every US is linked to its polygon on the map and to its periodisation, the system can produce a periodised plan by simply asking it. Pick a period, get a map of the units active in that period, with the units that survive into it automatically included. Repeat for the next period. The interpretive plan of the site, divided by phases, is a query, not a manual drafting exercise.

Thematic maps work the same way for any other dimension: by definition (cut, layer, structure, surface), by interpreter, by year, by material content of the layer. The map is a live view on the database.

5.3 ICCD-style sheets and printable catalogues

PyArchInit produces printable sheets in the layout used by the Italian heritage authority (ICCD) for the main record types — US, USM, RA, structures, periodisation. These are PDFs that can be delivered to a Soprintendenza as part of an excavation report.

Honest note on customisation: ICCD layouts are a moving target, and every Soprintendenza tends to ask for slightly different things — an extra logo, a specific header, a different ordering of fields, a slightly tweaked form. PyArchInit covers the standard ICCD layout reliably. Going beyond that, towards the exact preference of a specific Soprintendenza, is possible but requires customising the templates. Out of the box you get the standard; with effort you get the local variant. This is a real-world compromise that the project should plan for, not a defect to hide.

5.4 Quantifications on materials

Once the materials are entered, PyArchInit runs quantifications on them: counts, weights, NMI, distributions by class, by type, by US, by period. You decide which subset to quantify and on which field; the system aggregates. The output is a table that you can export and that updates automatically when you change the underlying records.

5.5 Cross-queries between context and content

Because everything is in a single database, you can ask the kind of questions that on paper take days: how many Hellenistic amphorae fragments came from contexts cut by Roman structures; which US of phase II yielded coins; which structures share at least one US dated to the same period. These are not built-in reports — they are queries that the data model makes possible. The system gives the archaeologist (or the GIS manager) the substrate to formulate them.

5.6 A real archive, not just a working file

When the excavation is over, what remains of PyArchInit is a database (Spatialite or PostgreSQL) plus the media folder. This is the archive: structured, queryable, portable, and independent from the version of the plugin that produced it. Years later, the same database can be reopened, re-queried, re-published, migrated to a new schema, or fed into a research project completely different from the one that generated it.

6. Beyond the core: analysis, animation, site management

The features in §5 form the **scientific backbone** of PyArchInit — what one might call the *stratigraphic core*. Around this core, the plugin has grown a set of tools that broaden its reach into landscape analysis, spatial statistics, animated visualisation, cartographic output, and operational management of an excavation. They are not the soul of the system, but they are the reason a project can run end-to-end inside PyArchInit instead of jumping between half a dozen disconnected tools.

6.1 Time Manager — replaying the stratigraphy in time

The Harris Matrix is, by nature, a static graph. The Time Manager turns the same stratigraphic order into something you can *replay*: a dial advances through the order of deposition, the map shows which units are visible at each step, units appear and disappear as the timeline moves, and the system can record the sequence as an animation or video. Useful for outreach, didactics, and as a sanity check on the periodisation: if the playback looks wrong, the periodisation usually is.

6.2 Make Your Map — cartographic outputs from a template

The QGIS canvas is good for working; a Soprintendenza wants a *plate*: title, scale, north arrow, legend, project code. Make Your Map provides a set of print templates (A4 / A3, portrait / landscape, with periodised variants) that consume the current view and produce a publication-ready PDF, PNG or SVG. It is a small feature with a large practical effect: the time from “finished interpretation” to “deliverable plate” drops from hours of layout work to minutes.

6.3 GeoArchaeo — geostatistics for archaeological data

Archaeological distributions — find density, layer thickness, geochemistry, anthropic indicators — are spatial signals. GeoArchaeo brings the standard geostatistics workflow inside the plugin: variogram modelling, kriging interpolation, machine-learning predictions on the spatial pattern, and the design of sampling strategies for future campaigns. Output is both a set of maps and a structured analytical report. The audience is landscape archaeology, predictive modelling, and any campaign that needs to decide *where to dig next* on more than intuition.

6.4 MoveCost — least-cost paths for landscape archaeology

How did people move across a territory? Least-Cost Path Analysis (LCPA) answers this question on a digital terrain model, with anisotropic cost functions that account for slope and direction. MoveCost packages the well-known cost models (Tobler, Llobera and others) behind a single dialog, runs them on a chosen DTM and start/end points, and returns the optimal paths as proper GIS layers ready to be queried, exported, and overlaid with the archaeological record. Previously it was buried inside the Site form; it is now a standalone analysis tool.

6.5 Gestione Cantiere — the operational side of an excavation

An excavation is also a workplace. Personnel, attendance, equipment, budget, volumes excavated (*computo metrico*) are normally kept in Excel sheets that have no connection with the scientific archive. Gestione Cantiere folds them into the same PyArchInit database: a dashboard with the operational KPIs, a personnel registry, an attendance log, an equipment inventory, a budget broken down by category, and DEM-based volume calculations. Exports for both PDF (for the funder) and CSV (for accounting) are built in.

Honest note: this module is *operational*, not scientific. It serves the site director and the project administration, not the stratigrapher. Including it in PyArchInit is a deliberate choice — it keeps a single archive instead of forcing a project to maintain two — but its lifecycle is distinct from the stratigraphic core and a project that does not need it can simply ignore it.

6.6 AI Query Database — asking the database in plain language

PyArchInit can be queried in natural language: the user types something like “*show all ceramic finds from the Roman phase of site X*”, the system generates the corresponding SQL, executes it on the database, and returns

the results, with an explanation of the generated query for those who want to learn from it. Three backends are supported: OpenAI (GPT-class models, online), Ollama (local models for sensitive datasets), and a fallback free API.

This is the only AI feature that PyArchInit ships in mature form today. The rest of the AI surface — matrix consistency checking, typological classification, automated quality control, retrieval over a CIDOC-CRM knowledge graph — is part of the frontier described in the closing postscript and is not claimed as an existing capability.

7. The compromises (the honest part)

Every tool comes with a price. PyArchInit is no exception, and this section makes its costs explicit so that the project can decide them with open eyes.

7.1 You have to follow the data model

PyArchInit is opinionated. It assumes the classic Italian stratigraphic tradition: US/USM, ICCD finds inventory, Harris Matrix, periodisation by phases. If a project uses a very different recording tradition — different unit names, different relation types, different sheet content — PyArchInit can be adapted, but it will not feel native. The system rewards those who work inside its assumptions.

7.2 Customisation is possible, but it is work

Adding a new field, a new form, a new export layout, a new ICCD variant is technically possible — the system is open source and modular. But it requires touching the code, the schema, or the report templates. There is no graphical configurator that lets a non-developer reshape the system. For a project of the scale of StratiGraph, this means that customisations should be planned, scoped and developed as explicit tasks, not improvised.

7.3 Some standards drift over time

ICCD schemas evolve, soprintendenze update their requirements, controlled vocabularies are revised. PyArchInit keeps up with this drift but with a delay: when a new ICCD release comes out, the corresponding mapping has to be updated in the plugin. Until that happens, exports follow the previous version. This is the nature of any tool tied to external standards, not a specific weakness.

7.4 The GIS is part of the tool

PyArchInit cannot be used outside QGIS. It is not a web application, it is not a standalone desktop program. The archaeologist who uses it must be at least minimally comfortable with QGIS: opening a project, managing layers, understanding coordinate systems. This is a feature, not a bug — PyArchInit gains everything from being in QGIS — but it is also a constraint to communicate honestly.

7.5 Field data entry is not always practical

On a real excavation, a laptop in the trench is often impractical: dust, sun, power, rain, time. The traditional answer is paper first, transcription later — with the errors that come with both steps. PyArchInit acknowledges this reality and is increasingly paired with companion tools (mobile apps, lightweight web interfaces, QField-style workflows, and the independent *pyarchinit-mini* initiative) to bridge the field-to-database gap. The main plugin is the post-processing and archival hub; the field tools are part of the surrounding ecosystem.

7.6 Multi-user is real, but needs a server

On a single laptop with SpatiaLite, PyArchInit works fine for one person. To have a team working on the same archive at the same time, a PostgreSQL/PostGIS server is required, with the usual server-side concerns: hosting, backups, access control, network. This is not a high bar, but it is a decision that has to be taken at project setup, not improvised mid-season.

8. What PyArchInit is not

An equally honest list, because the absence of features is often more useful to know than their presence.

- **It is not a 3D platform.** PyArchInit stores 3D models as attached media and links them to records, but it does not manipulate point clouds, meshes or photogrammetric models. That is the job of dedicated tools (Blender, CloudCompare, Metashape, and — within StratiGraph — the dedicated 3D components).
- **It is not a publication platform.** It exports data in formats that other systems can ingest (GeoJSON, GeoPackage, PDF, structured tables), but it does not host a public website of the excavation.
- **It is not a CAD.** Drafting precise architectural sections or detailed elevations is done in QGIS or in dedicated CAD/photogrammetry software; PyArchInit consumes those drawings, it does not produce them from scratch.
- **It is not a finished product.** It is a community project under continuous development. Features mature at different rates, some areas are very stable (US, matrix, ICCD sheets), others are more recent and evolve faster (materials specialisations, mobile companions, 3D integration).

9. What PyArchInit talks to

PyArchInit lives in an ecosystem. It is built to exchange data with the tools that an archaeological project actually uses.

- **QGIS** is the host environment: layers, projects, styles, print layouts are all native QGIS, fully reusable.
- **PostgreSQL / PostGIS** for team and institutional deployments, with the usual benefits of a real server database.
- **Spatialite** for single-user, offline, portable archives.
- **Graphviz** for the rendering of the Harris Matrix.
- **R** for the statistical analyses behind GeoArchaeo and MoveCost, and for any further bespoke analysis on materials and distributions through bridges that PyArchInit exposes.
- **Blender and 3D pipelines** for the visualisation and modelling of structures and finds, with media references kept in the PyArchInit archive.
- **yEd Graph Editor** for visual editing of Harris-Matrix-style stratigraphic graphs exchanged as GraphML.
- **Total Open Station** and total-station workflows for the import of survey data.
- **QField and mobile companions** (including the independent *pyarchinit-mini*) for field acquisition, with synchronisation back to the main archive.
- **External catalogues and open data portals** through export in GeoJSON, GeoPackage, CSV and structured tables.

10. One-page summary

What it is. A plugin that turns QGIS into an archaeological information system. Open source, free, maintained by an active community since 2005.

What it does. Keeps the excavation data — sites, stratigraphic units, relations, periodisation, structures, finds, materials, media — in a single database tightly linked to the cartography. From this database it produces the Harris Matrix, the periodised plans, the ICCD-style sheets, the quantifications and the structured exports. Around this scientific core it offers tools for temporal animation of the stratigraphy, cartographic plate output, geostatistical analysis, least-cost path modelling, operational management of the excavation, and natural-language querying of the database.

What it gives the archaeologist. A single source of truth, automatic checks on stratigraphic logic, deliverables compatible with national heritage standards, and an archive that survives the end of the excavation.

What it asks in return. Acceptance of an opinionated data model, basic comfort with QGIS, and the planning of customisations as explicit work rather than improvisations.

What it is not. A 3D tool, a web publishing platform, a CAD, or a finished commercial product.

What comes next. The closing postscript below frames the five archaeological problems that PyArchInit, today, still leaves on the table — and that the StratiGraph project sets out to address.

Where StratiGraph fits

PyArchInit, as described in the previous ten sections, is a working, mature system. After two decades of development it solves most of the day-to-day problems of an Italian-tradition excavation archive, and it does so in a way that an archaeologist can adopt without becoming a software engineer.

It still leaves five archaeological problems on the table. Not because the plugin is incomplete, but because these problems require infrastructure that lives at the European level, not at the level of a single QGIS plugin. The **StratiGraph** project sets out to address them. The list below is deliberately framed as *problem* □ *direction*: it describes the open issue and the kind of solution the project will pursue, without prescribing the specific tools, panels or versions that will deliver it.

Problem 1 — Persistent record identity

Today, when an archive is copied between institutions, or when two campaigns are merged, the local IDs collide and break. There is no way to say *“this US 100 is the same one I described five years ago in another database, in another country, with a different schema”*. The internal numbering of each PyArchInit installation is locally consistent but globally meaningless.

Direction. Assign every record a globally stable identifier from the day it is created, so the same row can be recognised, linked and cited across institutions and across time — a prerequisite for any aggregated European catalogue.

Problem 2 — Offline-first field documentation

Trenches do not have WiFi, and traditional sync solutions assume a connected environment. The current workflow loses data — typed once on paper, retyped in the lab, sometimes lost in the gap. Even when a laptop is present in the field, a temporary disconnection at the end of the day can mean an entire shift of records left in limbo.

Direction. A documentation workflow that *expects* to be offline. Local edits are queued and validated as soon as they are made; when connectivity returns, the queue is merged upstream with explicit replay and conflict resolution. The archaeologist sees a single archive; the underlying machinery handles the network.

Problem 3 — Stratigraphy □ 3D

The Harris Matrix is, by construction, a 2D abstraction of relations. The structures it describes — walls, tombs, building phases — are 3D objects, modelled separately in Blender or in photogrammetry pipelines. Today there is no shared vocabulary that lets a US in the matrix talk to its 3D counterpart, with the same identifier and the same paradata: who recorded it, under which licence, with which embargo.

Direction. An explicit, bi-directional bridge between the stratigraphic record and a 3D-aware extended-matrix model. Editing on one side reflects on the other; paradata travel with the entity, not with the file format.

Problem 4 — European semantic interoperability

PyArchInit speaks Italian standards (ICCD, US, RA, MA). Pan-European platforms and aggregators speak CIDOC-CRM, CRMarchaeo and adjacent ontologies. Today the export from one to the other is a manual mapping written ad hoc for each contribution; the same effort is re-done at every submission.

Direction. A maintained, transparent mapping between the PyArchInit data model and CIDOC-CRM, so that submission to an aggregator (Europeana, ARIADNE, the future Knowledge Graph of StratiGraph itself) is a configuration step, not a project.

Problem 5 — AI for quality and classification

The database can be queried in natural language (§6.6), but the AI does not yet *help* with the archaeological work itself. Detecting inconsistencies in the matrix, suggesting typological classifications for materials, flagging records

that fall outside the patterns of the rest of the archive — these are tasks where supervised AI can save weeks of work per campaign and, more importantly, catch errors that a tired human would miss.

Direction. A small set of focused AI assistants, embedded in the forms, trained on archaeological data, and always under the archaeologist's review. Not a chatbot, not a replacement: a second pair of eyes that never gets tired and is always honest about its confidence.

Closing

PyArchInIt will not be replaced by StratiGraph. It will be **extended**. The result, at the end of the project, will be a system that is at the same time a personal working environment for an archaeologist and a node in a European archaeological infrastructure — without forcing the archaeologist to choose between the two.

End of document.